12 Running Java in containers

This chapter covers

- Why container-driven development is important for the well-grounded Java developer
- The difference between an OS, a VM, a container, and orchestration
- Docker
- Kubernetes
- Practical guidance on running Java workloads in containers
- Performance and observability in containers

Docker (**https://www.docker.com/**) containers have become the de facto standard for packaging Java applications for deployment, and Kubernetes (**https://kubernetes.io/**) (k8s) is the most popular option for orchestrating those containers. Especially if you are deploying to any of the major cloud providers, you will need to know about these technologies and, more importantly, how Java behaves with them.

NOTE Although other container and container orchestration technologies exist, Docker and Kubernetes dominate the container and orchestration markets, respectively.

12.1 Why containers matter for a well-grounded developer

To better understand what containers are and why they are important for a well-grounded Java developer, we will look at the following:

- Host operating system versus virtual machines versus containers
- The benefits of containers
- The drawbacks of containers

12.1.1 Host operating systems vs. virtual machines vs. containers

Since the early days of computing, we've been introducing layers of abstraction between our software and the hardware that it runs on. Containers are another natural step in this progression. Let's take a brief tour through these layers to see how containers fix in.

Bare metal

Let us start by going right back to basics—a *bare metal machine* that has no host operating system installed. This bare metal machine represents a set of finite resources to whatever software might be installed on it, including CPU, RAM, hard disk, networking and so forth.

NOTE This concept of finite resources is a critical one to keep in your head. Far too often, developers are fooled into thinking that containers somehow give them magical infinite resources!

Always remember that underneath the host operating system, virtual machines, or containers is a piece of bare metal with *finite resources*.

Host operating system or type 1 hypervisor

In modern data centers, the bare metal machines have either a host operating system (e.g., Linux) or a Type 1 hypervisor (e.g., VMWare ESXi, Microsoft Hyper-V) installed on them. Hypervisor is the term for software that enables the creation and management of virtual machines. Hypervisors can exist at multiple layers of the stack. A Type 1 hypervisor is installed on bare metal and serves as a lightweight operating system, dedicating most of the machine's resources to the virtual machines it runs.

Whether running a traditional operating system or a hypervisor, this first layer is typically lightweight and doesn't do much more than look after security guarantees and allow for higher-level abstractions to be installed on top. That said, the host operating system does require some CPU, RAM, and networking to run.

Type 2 Hypervisors

If our bare metal has a traditional operating system such as Linux installed, then the next layer up is usually a Type 2 hypervisor. Whether Type 1 or Type 2, hypervisors are responsible for managing the resources of the underlying hardware for the virtual machines (VMs) with guest operating systems.

For example, a bare metal machine with 32 GB of RAM and a 16 core CPU with a Linux host operating system could run a Type 2 hypervisor, which in turn hosts four VMs, each running a Linux guest operating system with what appears to be 8 GB of RAM and 4 CPU cores each. Modern hypervisors normally don't take up much of the underlying resources to run themselves. If a Type 1 hypervisor is used directly on our bare metal, it's ready to run the next layer, virtual machines, without additional intervention.

Virtual machines

Each VM is fully self-contained. As far as the user is concerned, it has its own CPU, RAM, network, and disk resources. When you log on to a server in a production environment, chances are that you are logging on to a VM and not a bare metal server.

The self-contained VM also has its own operating system, referred to as the guest operating system. In the past, VMs paid a performance penalty to provide this isolated environment, but advances in the technology have removed many of those issues over the years.

Remember what we said about finite resources? Each virtual machine is just that—*virtual*. When the hypervisor isn't configured correctly, or virtual machines are given more resources than what physically exists, or are not dedicated to you (very common in cloud environments), you can have unpredictable performance.

Container engines

Prior to modern container engine technology, it was common to run a container engine on top of the guest operating system. This container engine could itself then run multiple containers.

This layer demonstrates one of the main differences between VMs and containers, because one of the container engine's key responsibilities is sharing access to a single operating system kernel across the containers it runs. This setup is much lighter than the VM model, where each instance has a full copy of its own operating system. This advantage, though, requires a lot of support from many different parts of the Linux kernel itself.

Containers

Finally we arrive at the container. You can think of a container as a custom-built, isolated environment in which to run an application. A container has a filesystem and runs at least one process. Although the processes in that container can all communicate with a kernel, many limits are imposed to keep the container separate from the rest of the world, including limits on memory, CPU, network (usage and visibility), and disk.

Inside containers, you run your Java application, data store, or other services that you require. Let's look at all of those layers of abstraction.

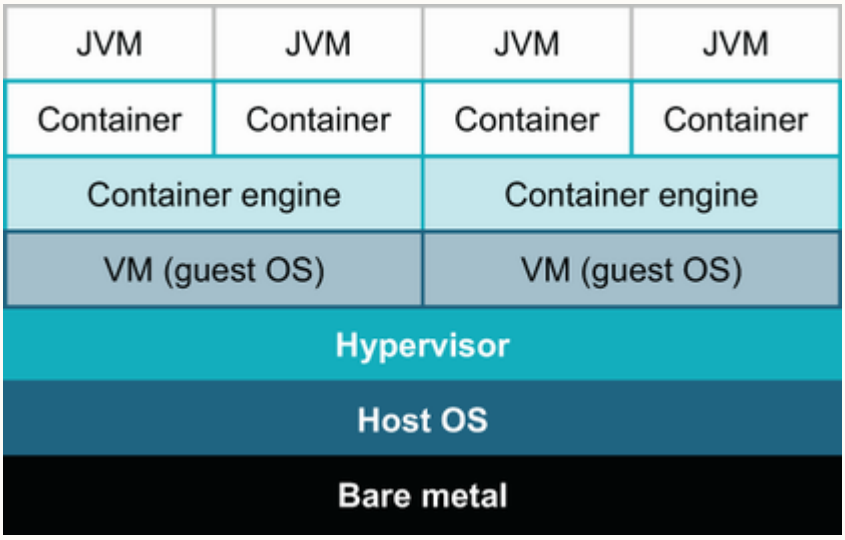


Figure 12.1 Target environment for Java applications

In Figure 12.1, the host operating system is the bottom layer of abstraction. The hypervisor is the next layer, followed by the container engine, the container, and the Java application. That seems a little over the top, doesn't it? In more pure container environments, it is, so in the past few years, you'll see dedicated container host machines, shown in figure 12.2, which remove the hypervisor and guest operating system layers.

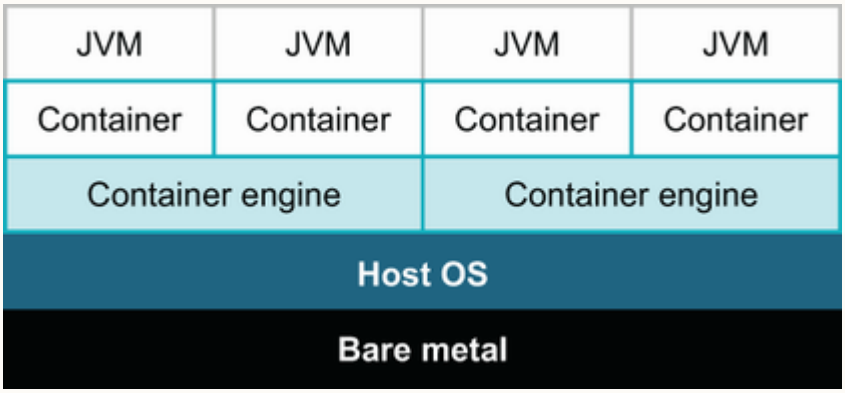


Figure 12.2 Target environment for Java applications on dedicated container engines

That's much better! That said, most developers aren't sure what their target environment looks like. The takeaway here is to make sure you check with your system administrators to understand exactly what your target environment looks like and how much of that bare metal finite resource is being allocated at each layer.

Despite all of the complexity within these layers of abstraction, as a Java developer, you will be focused mainly on the container as a deployment target, and this way of doing things has some important benefits.

2.1.2 Benefits of containers

With all the additional moving parts needed to run containers, why have they become the new standard for deployment? One of the key benefits is the ability of containers to apply limits, isolating individual running processes from each other. In the past, if you deployed two Java applications on the same host, there was a high probability that they could interfere with each other's performance—stealing too much CPU time or gobbling up more than their fair share of memory. Mitigations existed, but these ideas are baked into the fundamental layers of containers. In fact, having limits we can trust, in practice, allows us to more thoroughly use our computing resources, running more software on a host than would have felt safe before containers.

This isolation is so key that for the remainder of the chapter we'll illustrate the relationship between containers, hosts, and processes by showing nested rather than stacked images. Both ways of visualizing the relationship are valid, so don't be surprised to find both in the wild, depending on the context.

Containers have also ushered in a world of more consistent packaging for deployment. How you copied the bits for your application to the deployment environment, how you managed the operating system dependencies, and even how you managed process startup used to be all up for grabs. Containers provide an answer to all of that, making a gigantic pile of tooling and custom scripts unnecessary. They also provide insulation between our deployment environment and the container's contents. Our container engine doesn't have to care how we lay out the internals of our container—it just has to know how to start itself when asked. Packaging of container images is a key example of Infrastructure as a Service (IaaS), with a declarative, source-controlled description of layers in our system, which used to require careful, imperative construction.

A final benefit builds on that consistent packaging—the ecosystem that's developed around containers. Today almost any significant piece of software that you want to run has likely been packaged in a container already on Docker Hub or elsewhere. Vast READMEs of installation instructions or custom install scripts are now unnecessary.

But it can't all be upsides, right? What are the drawbacks to running in containers?

2.1.3 Drawbacks of containers

As it turns out, the first point that we listed as a benefit for containers—their built-in isolation—is actually one of the difficulties in using them. The job of containers is keeping the world inside the container separate from the outside world, and that outside world includes you, the developer. Many of the techniques and tools that you commonly use

outside of containers may require special handling and configuration with a move to containers.

This can be especially true when trying to factor containers into your local development processes. Longer builds and time spent shuffling huge container images around may not always feel worth it.

And although containers introduce a consistent interface for how we package and start our application, real-world deployment isn't always simple. For example, an application that expects full access to the disk on its host may require configuration to make the required files visible to the container. If a set of processes communicate with each other, separating them into containers will need explicit configuration for how they can access each other. Capturing and applying this sort of configuration is a key task for orchestrators such as Kubernetes. Be aware, however, that Kubernetes, which we'll examine briefly in this chapter, is a topic fit to fill books, and the ecosystem continues to develop rapidly.

Although containers are becoming entirely mainstream, the well-grounded developer knows how to examine the trade-offs to find the right balance for their system. Let's start to see how to use these tools, so we can get a feel for where they fit.

12.2 Docker fundamentals

Though many of the constituent technologies that make up containers existed earlier, Docker introduced convenient tooling and abstractions that catapulted containerization to the mainstream. Let's take a look at two central pieces of functionality that Docker gives us—building images and running containers—and see how we interact with those as Java developers in practice.

12.2.1 Building Docker images

A Docker container is launched from an *image*. An image is essentially a snapshot that captures all the filesystem dependencies needed to run a piece of software. An image includes native libraries, languages runtimes, tools, and, most importantly, a specific version of your software to run.

The `Dockerfile` is typical format for capturing the set of steps to build an image. The simplest image possible, one which is entirely empty, looks like this:

```
FROM scratch
```

We build the image using the `docker build` command as follows:

```
$ docker build .

[+] Building 0.1s (3/3) FINISHED
 => [internal] load build definition Dockerfile    0.0s
 => => transferring dockerfile: 55B                0.0s
 => [internal] load .dockerignore                  0.0s
 => => transferring context: 2B                    0.0s
 => exporting to image                             0.0s
 => writing image sha256:71de1148337f4d1845be0...  0.0s      (1)

Use 'docker scan' to run Snyk tests against images to find vulnerabilities
and learn how to fix them
```

(1) The sha256 ID 71de114... uniquely identifies the resulting image. We'll see how to give things a more friendly name in a moment.

Of course, an empty image isn't much use. In practice, many base images exist with useful software already installed. The default source of these base images is Docker Hub (https://hub.docker.com/). We'll talk more later about selecting the right Java base image, but for now, let's start building an image off an Eclipse Temurin version of OpenJDK provided by Adoptium. We'll specifically choose the `eclipse-temurin:11` image as show here, which contains the latest version of Java 11:

```
FROM eclipse-temurin:11
RUN java -version
```

By default, recent versions of Docker will dynamically hide output when building in an interactive terminal. We'll use `--progress plain` here to get a clearer picture of what's happening:

```
$ docker build --progress plain .

=1 [internal] load build definition from Dockerfile                (1)
=1 sha256:261a2389333859f063c39502b306e984de49700a9...
=1 transferring dockerfile: 36B done
=1 DONE 0.0s

=2 [internal] load .dockerignore                                   (1)
=2 sha256:909e36a5a9cd7cc4e95e7926f84f982542233925d...
```

```
=2 transferring context: 2B done
=2 DONE 0.0s

=3 [internal] load docker.io/library/eclipse-temurin:11                    ❶
=3 sha256:6a73b62137bbf64760945abf21baf23bf909644cf...
=3 DONE 0.5s

=4 [1/2] FROM docker.io/library/eclipse-temurin:11...                      ❷
=4 sha256:f225b618d7ad96bd25e0182d6e89aa8e77643f42f...
=4 CACHED

=5 [2/2] RUN java -version                                                 ❸
=5 sha256:556476b43b8626a27892422f8688979c4ba1e6029...
=5 0.38 openjdk version "11.0.13" 2021-10-19
=5 0.38 OpenJDK Runtime Environment Temurin-11.0.13+8 (build 11.0.13+8)
=5 0.38 OpenJDK 64-Bit Server VM Temurin-11.0.13+8 (build 11.0.13+8)
=5 DONE 0.4s

=6 exporting to image                                                      ❶
=6 sha256:e8c613e07b0b7ff33893b694f7759a10d42e180f2...
=6 exporting layers 0.0s done
=6 writing image sha256:9796a789e295989cec550f... done
=6 DONE 0.0s

Use 'docker scan' to run Snyk tests against images to find vulnerabilities
and learn how to fix them
```

❶ Internal steps Docker takes when preparing to build our images

❷ Retrieving our requested base image

❸ Our RUN command is executed during the build, and we can see its output.

You may note that this code takes longer to run, at least the first time, because Docker has to download the relevant base image from Docker Hub. The `RUN` command we've added introduces our own new step on top of that base image. `RUN` can execute any valid command in the container environment. If the command alters the filesystem, those changes are captured as part of our final image. This example doesn't actually alter the filesystem, but `RUN` is frequently used to download files (e.g., via `curl`), install operating system packages with standard package managers, or make other local modifications.

We can see another important part of building Docker images if we run the same build command again without touching the `Dockerfile` like this:

```
$ docker build --progress plain .

=1-4 excluded for length...

=5 [2/2] RUN java -version
=5 sha256:556476b43b8626a27892422f8688979c4ba1e602907a09d62a39a2
=5 CACHED                                                        (1)

=6 exporting to image
=6 sha256:e8c613e07b0b7ff33893b694f7759a10d42e180f2b4dc349fb57dc
=6 exporting layers done
=6 writing image sha256:9796a789e295989cec5550fb3c17bc6c1d9c0867  done
=6 DONE 0.0s
```

(1) Docker informs us when it skipped a step because the result was cached.

Each of the leading commands (like `FROM` and `RUN`) in the `Dockerfile` creates what is called a *layer*. Because these commands can often be time-consuming, those layers are cached, and Docker does its best to avoid unnecessary work.

Now that our container has a Java environment, we can run our own code there. We'll create a simple Java file alongside our `Dockerfile` in `HelloDocker.java`. To keep things easy to start, we'll use the Java single-file execution to run this instead of putting a full build together yet. The basic code looks like this:

```
public class HelloDocker {
  public static void main(String[] args) {
    System.out.println("Hello Docker!");
  }
}
```

We can then instruct the Docker build to include this file in our image and set the default command for containers running this image as follows:

```
FROM eclipse-temurin:11
RUN java -version

COPY HelloDocker.java .          (1)
```

```
CMD ["java", "HelloDocker.java"]  ❷
```

❶ Copies our file to whatever that current working directory Docker has set

❷ Sets the default command for the image. Note each command-line argument is in its own separate string.

`COPY` (and the more complex `ADD` command) take files from our local build environment and puts them in our container. `ADD`, in particular, has a lot of options, including fetching from remote source and auto-extracting TAR files, but generally, you'll be better off with a simple `COPY` where you can.

`CMD` points us to the next stage in an image's lifecycle. We aren't just building these images for fun—we want to run the software we're configuring in them. As mentioned earlier, each image has a unique SHA256 identity, but those are cumbersome to work with and change every time you build. Before we get to running our image, let's tag our image with an easier name, as shown next:

```
$ docker build -t hello .

... Previous build steps excluded for length

=8 exporting to image
=8 sha256:e8c613e07b0b7ff33893b694f7759a10d42e...
=8 exporting layers done
=8 writing image sha256:666fdc7613189865b9a5f2... done  ❶
=8 naming to docker.io/library/hello done               ❷
=8 DONE 0.0s
```

❶ Our image's SHA256 identity

❷ The tag we've applied to the final image

Our `hello` image is available only locally at this point, but we've already seen via the `FROM` line that images can be shared. This is accomplished through what's known as a *container registry*. When we asked for the `eclipse-temurin:11` base image, Docker defaulted to looking for that image on Docker Hub (**https://hub.docker.com/**). Other container registries exist, and, in fact, they can be run internally for hosting your application images.

You can push and pull images via the `docker push` and `docker pull` commands, respectively, as shown in the following code. If working with a non-default registry, that name is given before the image and tag name:

```
$ docker pull k8s.gcr.io/echoserver:1.4     (1)
1.4: Pulling from echoserver
6d9e6e7d968b: Pull complete
...
7abee76f69c0: Pull complete
Digest: sha256:5d99aa1120524c801bc8c1a7077e8f5ec122ba16b6dda1a...
Status: Image is up to date for k8s.gcr.io/echoserver:1.4
k8s.gcr.io/echoserver:1.4
```

(1) k8s.gcr.io is the registry domain, echoserver is the image name, and 1.4 is the tag.

If the registry requires authentication, you may have to use `docker login` before proceeding. Publicly available images on Docker Hub don't require that step, though.

There's a lot more to building good Docker images, and we'll cycle back to some of those topics later. But first, let's see how to turn these images into running containers.

2.2.2 Running Docker containers

For all the hype and discussion you hear around Docker and containers, the central idea is simply being able to execute a well-defined process in a strictly controlled environment. The environment is largely defined by the image that we construct. Docker allows us to run a container with the `docker run` command, as shown here:

```
$ docker run hello
Hello Docker!
```

In this command, Docker created a new filesystem based on our image, applied limits and controls (such as CPU and memory), and then started the default process defined by `CMD`. Our program outputs a message and exits, but it could as easily have started a a server and kept running indefinitely.

In figure 12.3, we can see the `java` process we listed in the `CMD` for the image. Remember that the host shown here may in fact hide many additional layers before you reach the bare metal machine.

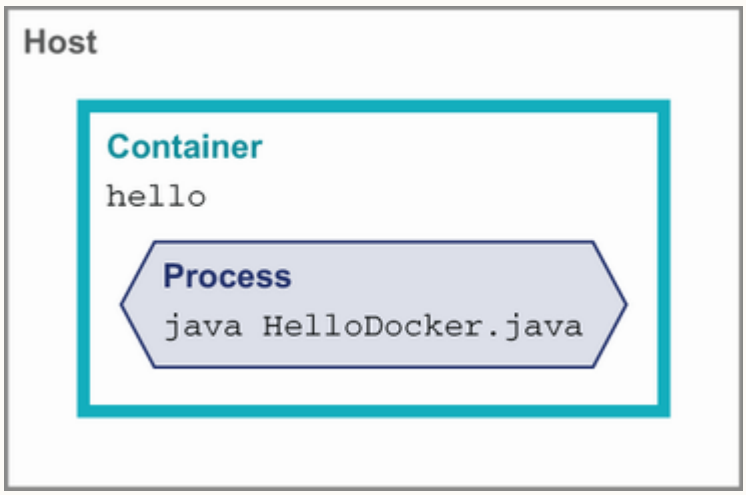


Figure 12.3 Running a basic container

Our `CMD` defines only the default command for starting the container. We can ask Docker to run the image with any alternative command we wish. We mentioned earlier that the container has a working directory, much like your interactive terminals do. We can ask the container what that path is with the `pwd` command as follows:

```
$ docker run hello pwd
/
```

As Figure 12.4 shows, when we run an alternative command to start the container, the default `CMD` process is nowhere to be seen.

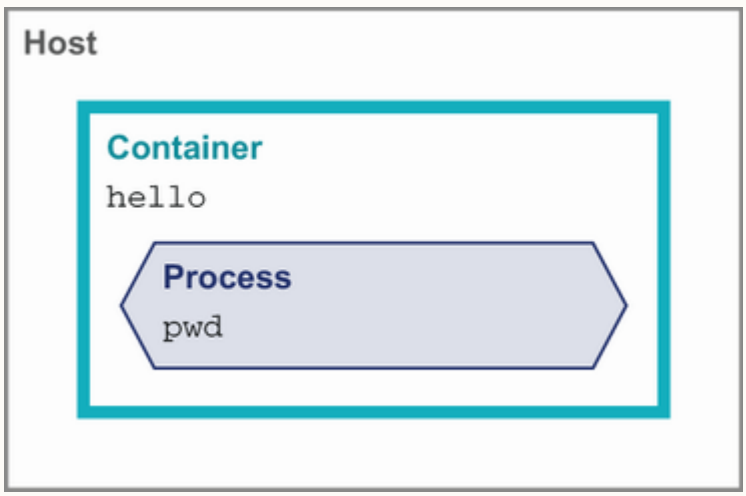


Figure 12.4 Running an alternative command in a container

We can write configuration into our image via files, but it's often desirable to allow defining them at runtime. One of the principles from the Twelve-Factor App (**https://12factor.net/**), an influential set of ideas about running software such as containers, is defining configuration via the *environment* so the same built resource (in our case, the image) can be deployed to new destinations without changing code.

As shown in the following code snippet, we can alter the environment variables within our container when we start it using the `-e` flag, which may be passed multiple times. In our application code, these variables may be read via standard means such as the `System.getenv()` method:

```
$ docker run -e MY_VAR=here -e OTHER_VAR=there hello env      (1)
PATH=/opt/java/openjdk/bin:/usr/local/sbin:/usr/local/bin:...
HOSTNAME=f25762652561
MY_VAR=here                                                   (2)
OTHER_VAR=there                                               (2)
LANG=en_US.UTF-8
LANGUAGE=en_US:en
LC_ALL=en_US.UTF-8
JAVA_VERSION=jdk-11.0.13+8
JAVA_HOME=/opt/java/openjdk
HOME=/root
```

(1) Runs the standard env command to see our container's environment

(2) Our full environment variable list

Let's discuss one last technique before we look at more realistic approaches to building our Java applications in containers: interactively running an image. We've seen changing the default command to run in the container. We can use that same ability to start a shell such as `bash` in the container for additional debugging. This requires extra flags to `docker run`—specifically `-i`, so `STDIN` is attached for our input to reach the container, and `-t`, so the container starts an interactive TTY for us, as shown here:

```
$ docker run -it hello bash
root@b770c2ac829c: ls *.java    (1)
HelloDocker.java
root@b770c2ac829c:
```

(1) Interactively enters shell commands to inspect the container

This lets us see the world precisely as our deployed applications will within the container.

It's all well and good to copy a hello world single-file app into a container, but let's look now at more realistic approaches to using Docker and Java together.

12.3 Developing Java applications with Docker

In this section, we'll tackle a variety of practical considerations for developing Java applications with Docker. We'll start with looking a bit deeper at our JVM base images and how to build our images. From there, we'll dig into various considerations about configuring, running, and debugging our containers. Our container has to get a JVM from somewhere, which brings us to the topic of choosing a base image.

12.3.1 Selecting your base image

There isn't a single answer to what the "right" base image is for running your JVM application. Determining what image works for you requires considering the following:

- What vendor do I want?
- What operating system do I want inside my container?
- What system architectures do I need to run on?

The choice of vendor also includes a number of factors that may influence your choice (we discussed this briefly back in chapter 1), including the following:

- Support availability and contracts
- Security update policies and timeliness
- Special considerations for cloud deployment—Microsoft Build of OpenJDK for Azure, Amazon Corretto for AWS

The cloud vendor–specific builds, while based on OpenJDK, may include performance and other enhancements that are beneficial in that vendor's cloud. They also may have additional support and release-frequency benefits.

Most vendors provide support on more than one operating system in their containers. It's common to see Debian, Ubuntu, or Alpine, along with some other Linux variants. The choice of operating system largely dictates what package manager is used to install native dependencies and what additional tooling is available within the container. If your requirements don't dictate a specific operating system, keeping to the more mainstream options like Debian/Ubuntu often avoids difficulties in finding and updating packages.

NOTE Take particular care with Alpine Linux. Until very recently, no official images for Java on Alpine existed. You should check with your Java vendor and make sure that they provide images for Alpine.

If you need to run on an operating system that isn't directly shipped by the vendor, don't despair. In these cases, you can build an image yourself that will use the system's typical package management to install the JDK manually. Remember, base images and our Docker builds are just about getting the right bits in the container's filesystem. There's often more than one way to get to that end result you're after.

A final note is on system architecture for images. It is becoming increasingly common to run on ARM-based chips, especially in the cloud. Although this has performance advantages, be aware that you'll need images that are built specifically for that architecture. If you need to run across architectures, you may end up having to build and publish multiple images, but the Docker tooling supports this well already.

2.3.2 Building an image with Gradle

As we saw in chapter 11, any sizeable Java project will benefit from using a consistent build tool. For demonstration purposes, we'll walk through how to construct an image based on a Gradle build, but a similar Maven version is available in the resources.

At a minimum, our image needs to contain all our application's JARs (or class files) and all the dependencies for our classpath. For our sample, our application takes a dependency on `org.apache.commons:commons-lang3`, as shown next:

```
plugins {
  application
  java
}

application {
  mainClass.set("com.wellgrounded.Main")
}

tasks.jar {
  manifest {
    attributes("Main-Class" to application.mainClass)
  }
}

repositories {
  mavenCentral()
}

dependencies {
```

```
  implementation("org.apache.commons:commons-lang3:3.12.0")
}
```

We need a slightly different command from our usual `build` or `assemble`, but Gradle's defaults have what we need wrapped up via `installDist`, as shown here:

```
$ ./gradlew installDist
```

The simplified build result from this command follows:

```
build
└── install
    └── docker-gradle
        ├── bin
        │   ├── docker-gradle
        │   └── docker-gradle.bat
        └── lib
            ├── commons-lang3-3.12.0.jar
            └── docker-gradle.jar
```

We could just take the JAR files and run from them in the container, but Gradle has created some helper scripts for starting our application. Let's take advantage of those next:

```
FROM eclipse-temurin:17-jdk

RUN mkdir /opt/app                          ❶
WORKDIR /opt/app/bin                        ❷

COPY build/install/docker-gradle /opt/app/  ❸

CMD ["./docker-gradle"]                     ❹
```

❶ Ensures the directory exists for us to copy results into

❷ Gradle's startup scripts expect the working directory to be bin, so set that as the default location for Docker to start.

❸ Copies the entire install results tree into the container

❹ The default command to run is now the startup script from Gradle.

You can find out more about Gradle's start scripts in the documentation for the Application plugin (see **http://mng.bz/yvxJ**).

This approach assumes that we have an appropriate JDK installed locally to build with Gradle before copying the results into our image. Up next, we'll see how we can wrap that up entirely in Docker.

2.3.3 Running the build in Docker

A key promise of containers is the ability to create an isolated, repeatable environment for our software to run in. This is a huge advantage for deploying our services, but it doesn't stop there. A classic problem for many projects is setting up a local environment for development. If you've ever plowed through a README with step after step of installs, making sure you got the right versions of everything, you know this pain. Containers can help us escape this. Let's examine how we can alter our build to leverage that isolation.

Our `Dockerfile` to date has involved only the one resulting image that we're trying to construct. But Docker allows us to define multiple images in the same file and, most important, copy between them. With this ability, we can construct an image in which to build our application—fully removed from whatever JDKs our local system has—and then copy the results into our deployment image. This has advantages for both security and image sizes.

This process is referred to as a *multistage build*, and you can see it in action when a `Dockerfile` has multiple `FROM` statements. `FROM` lines that are just intermediate stages of the build also include an `AS` keyword to name them for later use in the `Dockerfile`, whereas our main resulting image is left as before, as shown here:

```
FROM eclipse-temurin:17-jdk AS build                    ❶

RUN mkdir /project                                      ❷
WORKDIR /project

COPY . .                                                ❸

RUN ./gradlew clean installDist                         ❹


FROM eclipse-temurin:17-jre                             ❺

RUN mkdir /opt/app
```

```
COPY --from=build \                          ❻
    /project/build/install/docker-gradle-multi \    ❻
    /opt/app/                                ❻

WORKDIR /opt/app/bin
CMD ["./docker-gradle-multi"]
```

❶ Our container to run the compile in, named build

❷ Creates a location for our source code and sets it as the default working directory

❸ Copies our full project into the container

❹ Builds our application (in this case, the Gradle version) as before, locally

❺ Our deployment image can now just use the JRE, which is much smaller.

❻ COPY --from=build takes files from our build image instead of the local filesystem.

Now our continuous integration environment needs only Docker, not a JDK installed, to be able to build our application for deployment. As figure 12.5 shows, all the necessary components for the build remain fully within the containers.

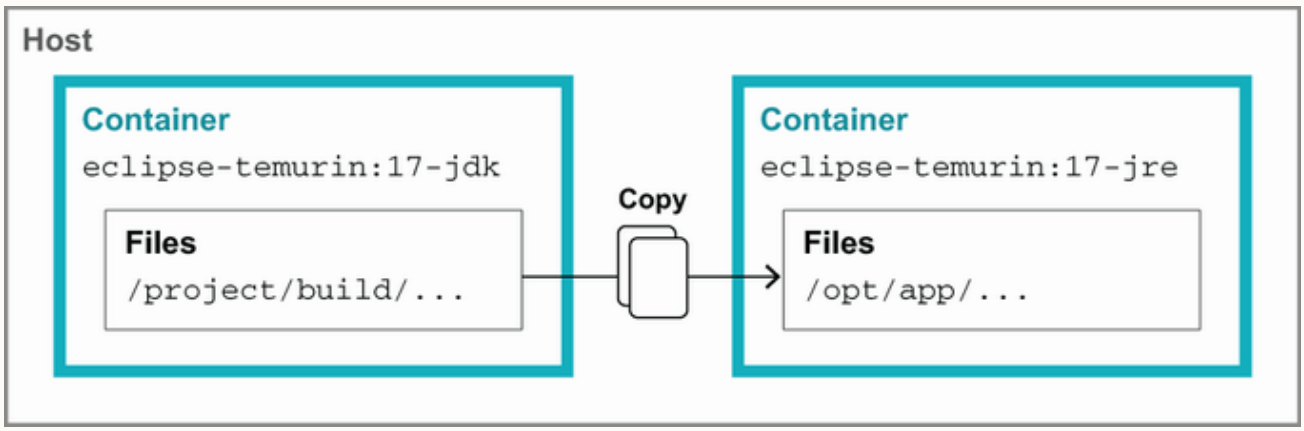


Figure 12.5 Multistage builds in Docker

It's worth calling out that this is close to the minimal setup for this type of build, but it has some downsides around build times. As we mentioned, each Docker command creates a layer that is cached, but those caches can be invalidated unnecessarily if we're not careful.

One source of such cache breaking in our current `Dockerfile` is where we copy the full project directory into the container. Any file change, no matter how small, will invalidate the `COPY . .` line, and we have to run everything after that fresh. It's possible that some local files don't matter to our build, though—for instance, our git history, IDE files, and local build output really don't need to end up in our build container image. Fortunately, we can exclude such files from Docker's consideration by placing a `.dockerignore` file alongside our `Dockerfile`. The format is simple and may be familiar if you've worked with `.gitignore` files before. As shown in the next code snippet, each line expresses a pattern (standard shell wildcarding allowed) that Docker should ignore when finding files to copy:

```
.git
.idea/
*.iml
*.class

# Ignore build folders
out/
build/
target/
.gradle/
```

A second more subtle issue is with our Gradle wrapper. If we watch the output when running the build, we'll see that it spends a while on startup downloading the proper distribution. Because our containers start without any of Gradle's local caching, this download repeats each time we run.

Avoiding this repetition requires breaking out a first execution of Gradle into a separate set of layers which happen before copying our full project into the container, as shown next. We want to copy only the minimum required for Gradle to run its download, so this layer's cache breaks only if we alter our Gradle wrapper (e.g., updating the version):

```
COPY ./gradle ./gradle                ❶
COPY ./gradlew* ./settings.gradle* .  ❶
RUN ./gradlew                         ❷

COPY . .                              ❸

RUN ./gradlew clean installDist
```

❶ Copies just enough Gradle configuration to run

❷ Running ./gradlew alone forces the distribution download, now cached in its own layer.

❸ Our build carries on as before, with our COPY likely refreshed every time (assuming our code changed).

This is just the beginning of the sorts of optimizations that can be applied in the construction of your container images. The key to take away is carefully considering what belongs in each layer. If parts of your system will change at different paces, giving them separate layers may be beneficial.

We've shown a fairly raw approach to building Docker images. As you might expect, a ton of plugins for both Maven and Gradle exist if you want to wrap up this functionality and not hand-code your Dockerfiles. There are even options like Jib (**https://github.com/GoogleContainerTools/jib**), which avoid using the Docker tools at all. All of these are useful, but a well-grounded developer is aided by understanding more deeply how containers are built, even if they get help doing it day to day.

2.3.4 Ports and hosts

Along with providing our application with its own isolated filesystem, containers also do the same for the network. With our sample application, let's imagine that we added code to run a standard HTTP server, for example, the basic one provided in the JDK at `com.sun.net.httpserver.HttpServer`. If we `docker run` our container, we'll find that there's no way to call that HTTP endpoint.

To address this, we need to ask Docker to make a port available to us. We can do this directly by adding to our run command as follows:

```
$ docker run -p 8080:8080 hello
```

`-p` takes a pair of ports separated by a `:`. The first value is the port that we want available *outside* the container. The second value is the port that our software *inside* the container is listening on. If we go to another terminal (or a web browser) we can see it working, as shown here and in figure 12.6:

```
$ curl http://localhost:8080/hello
```

```
Hello from HttpServer
```

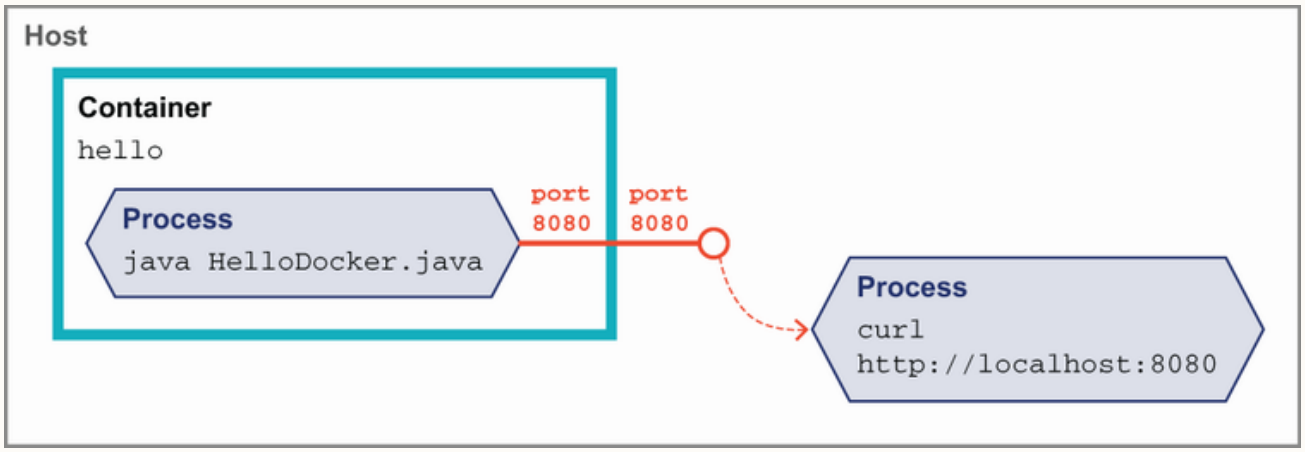


Figure 12.6 Exposing a port in Docker

As you might expect from the format and Figure 12.6, these two port values don't have to match. If we instead run with this command-line:

```
$ docker run -p 9000:8080 hello  (1)
```

(1) Port 9000 will be visible outside the container, connected to port 8080 on the process inside the container.

we'll now see a good response on port 9000, whereas 8080 is no longer accessible, as shown here:

```
$ curl http://localhost:9000/hello
Hello from HttpServer

$ curl http://localhost:8080/hello
curl: (7) Failed to connect to localhost port 8080: Connection refused
```

Exposing ports is such a fundamental part of how containers are deployed that the `Dockerfile` allows us to note the ports our image is expected to provide, like this:

```
EXPOSE 8080
```

If we set this, `docker build` again, and run without the port switches, you might be surprised to find that Docker doesn't default to making the `EXPOSE` ports available. However, if you provide the `-P` switch by itself (note the uppercase and lack of argu-

ments), Docker will bind each `EXPOSE` port in our image to a random, or *ephemeral*, port. Because we can't guess what port will be assigned, we need a new command to peek and find our ephemeral port. This is done with `docker ps` as follows:

```
$ docker run -P hello

... In another terminal, some columns trimmed...
$ docker ps
CONTAINER ID   IMAGE     COMMAND           PORTS
94d7f125caad   hello     "./docker-gradle"   0.0.0.0:55031->8080/tcp
```

The value `0.0.0.0:55031->8080/tcp` tells us that port 55031 outside the container is bound to port 8080 inside.

This ephemeral port business may seem like an annoyance at first, particularly when testing, because the port shifts around. But it's actually a critical feature when running containers in production. Imagine that you have a host that you want to fully utilize running many different Java containers. Each of those applications may want to run using the same port, but the host can assign that port only once. Although it requires additional coordination in other parts of the system, assigning ephemeral ports allows the containers to keep their simpler view of the world—"I run on 8080"—while still coexisting in a broader, more complex environment.

That gets us set to talk to our application when running it in a container locally. But what about the other direction—when our container needs to reach out to another services such as a database?

When we're running in production, it's a good practice to explicitly configure locations of services and use normal load balancing and DNS to reach them. These can be injected into containers via environment variables or other service discovery systems, but the key is that you don't assume where resources are located in relation to your container.

But this gets much harder when working locally, because a normal developer setup isn't going to have that same sort of infrastructure available. If you're using Docker for Mac or Docker for Windows, you can use the name `host.docker.internal` inside your containers, which automatically points to your host machine. Docker for Linux can have the same set on container startup with the `--add-host host.docker.internal :host-gateway` flag. In these cases, if your application is set up to receive such locations via environment variables, you can point your container to that hostname.

If this doesn't work for your given environment, an IP address for your host exists inside the container. Commands such as `sudo ip addr show` may give you hints on the location, but this gets tedious in a hurry.

Containers have a lot of networking options that are beyond the scope of this book, but some of them that can help us with this exact issue are used by a tool called Docker Compose. Let's see how containers can help us locally solve our external resource access problems.

2.3.5 Local development with Docker Compose

Much like the dreaded install list for a new project, it's common for applications to also require multiple other services at runtime as well. Maybe you've got a database, a cache, a NoSQL store, or even other custom applications, which all have to be up and running for your application to work locally.

Docker Compose is a tool for declaring and running sets of containers. It lets us capture the precise set of services and start them together. It also manages saving state for these containers so we can stop and restart without having to do everything from scratch.

If this sounds similar to orchestration tools like Kubernetes, you're not wrong. There is overlap in the container management aspects of both tools. However, Docker Compose is aimed at running on a single machine, which excludes it as a reasonable choice for many production environments.

NOTE Docker Compose was originally a separate tool, but it has been integrated as another command within `docker` itself. If you see information on the internet suggesting running `docker-compose`, you can just replace the `-` with a space these days.

By default we describe our configuration in a file called `docker-compose.yml`. For a start, let's tell Docker Compose about our application as follows:

```
version: "3.9"        ❶
services:
  app:                ❷
    build: .          ❸
    ports:
      - "8080:8080"   ❹
```

❶ The Docker Compose file version

❷ Declares a service to run called app

❸ Instructs Docker Compose to run a typical docker build in the current directory to generate the image for this service

❹ Port declarations, like on our manual docker run before

We run this at the command line with `docker compose up`. This will show our familiar build output as it starts, and then some new output as it starts our container, as shown next:

```
[+] Running 2/2
 - Network docker-gradle_default  Created                    0.1s
 - Container docker-gradle-app-1  Created                    0.1s
Attaching to docker-gradle-app-1
docker-gradle-app-1  | (Howdy,Docker)
```

Our `docker-compose.yml` can contain multiple services, as we'll see in a moment. The output from each gets prefixed with a name to distinguish them, by default based on our current directory and the service name, so ours is `docker-gradle-app-1`.

Let's say that our application needs a Redis instance. We add that as a new key we'll call `redis` under the `services` key like this:

```
version: "3.9"
services:
  app:
    build: .
    ports:
      - "8080:8080"
  redis:
    image: "redis:alpine"    ❶
```

❶ The redis:alpine image from Docker Hub

Now when we run, Docker Compose will pull the `redis:alpine` image and start it with our application container. Figure 12.7 and the next code sample illustrate these containers running in relation to each other:

```
[+] Running 7/7
 - redis Pulled                          5.0s
   - 59bf1c3509f3 Pull complete          1.2s
   - 719adce26c52 Pull complete          1.2s
[+] Running 2/2
 - Container docker-gradle-redis-1  Created    0.2s
 - Container docker-gradle-app-1    Created    0.0s
Attaching to docker-gradle-app-1, docker-gradle-redis-1
docker-gradle-redis-1  | # oOOOo Redis is starting...      (1)
docker-gradle-redis-1  | # Redis version=6.2.6, ...        (1)
docker-gradle-redis-1  | * monotonic clock: POSIX ...      (1)
docker-gradle-redis-1  | # Warning: no config file...      (1)
docker-gradle-redis-1  | * Running mode=standalone, ...    (1)
docker-gradle-redis-1  | # Server initialized              (1)
docker-gradle-redis-1  | * Ready to accept connections     (1)
docker-gradle-app-1    | (Howdy,Docker)                   (2)
```

(1) Redis container output

(2) Our application container output

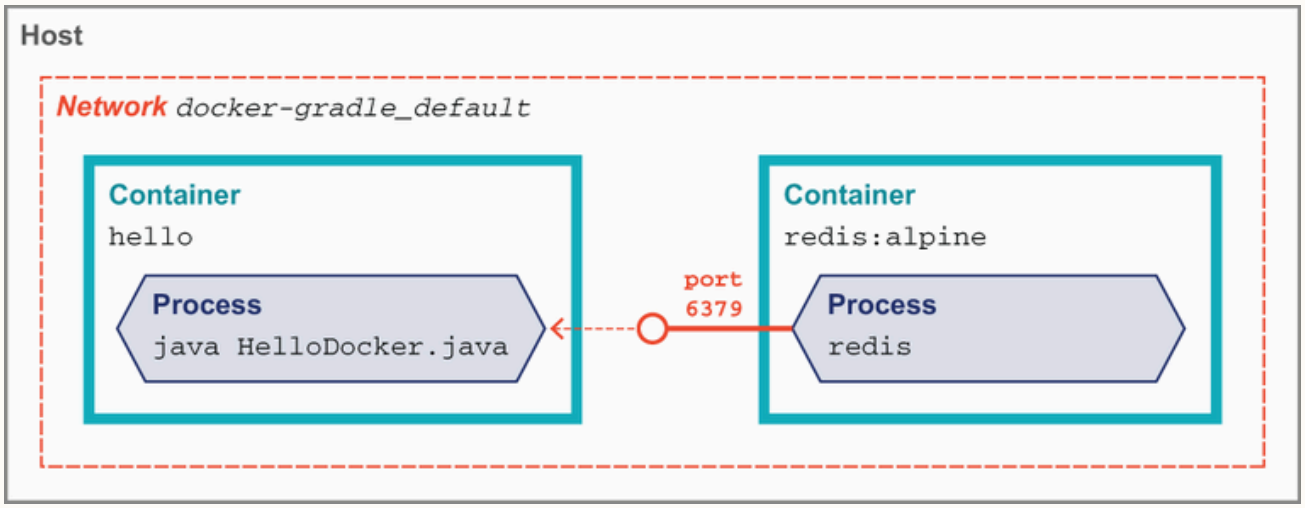


Figure 12.7 Docker Compose containers running

This is already convenient—we can get the precise right version of our databases and other external services locally without manual installation. But Docker Compose brings another helpful feature to avoid many of the networking struggles we saw before. During initial startup, there was a message reading `Network docker-gradle_default Created`. This informed us that Docker Compose had created a new, separate network namespace `docker-gradle_default`. This network is shared between all the services that Docker Compose has started for us. Even better, each service name we spelled out in our

`docker-compose.yml` — `app` and `redis` —shows up like a real host name inside all of the containers.

If we've designed our application with the Twelve-factor principles and pass in the location of Redis via an environment variable, we can configure this entirely in the `docker-compose.yml`, as shown here:

```
version: "3.9"
services:
  app:
    build: .
    ports:
      - "8080:8080"
    environment:
      REDIS_URL: redis://redis:6379   (1)
  redis:
    image: "redis:alpine"
```

(1) The first redis is the URL scheme, and the second redis is the hostname.

This just scratches the surface of Docker Compose. All the common options for controlling `docker run` can be set in `docker-compose.yml`, and it's a great way to smooth ramp-up on local development.

2.3.6 Debugging in Docker

When our software isn't behaving, sometimes we need to peek inside the boundaries that our containers set up for us. Earlier we met `docker ps` for determining the port that our container was exposing. `docker ps` provides us more information than just that, though. In particular, by default, a container is given a handy, randomly generated name that it can be referred to by, as shown next:

```
$ docker ps
CONTAINER ID   IMAGE    COMMAND          ...  PORTS     NAMES
c103de6e6634   hello    "./docker-gradle" ...  8080/tcp   vigilant_austin
```

This container can be referred to as `vigilant_austin`. If you want to avoid having the name change every time your container runs, you can control this with the `--name container-name` parameter on `docker run`. You'll want to couple that with `--rm` to remove

the container when it exits; otherwise, the name won't be available for reuse the second time you run.

Having the name of the container allows us to take other debugging steps. With `docker exec`, we can execute commands in the running container. As we saw with `docker run -it` before, we can even get an interactive shell inside the container, as shown here, assuming it has `bash` or something similar installed:

```
$ docker run --name hello-container --rm hello

# In another terminal start a shell in the container
$ docker exec -it hello-container bash

root@18a5f04bb4c8: ps aux
USER PID %CPU %MEM COMMAND
root  1 1.6  1.9 /opt/java/openjdk/bin/java -cp /opt/app/lib/docker-gradle
root  37  0.1  0.1 bash
root  47  0.0  0.1 ps aux
```

It's important to remember that `exec` doesn't start a new container—it attaches to an existing one. Figure 12.8 shows how the processes coexist within the single container.

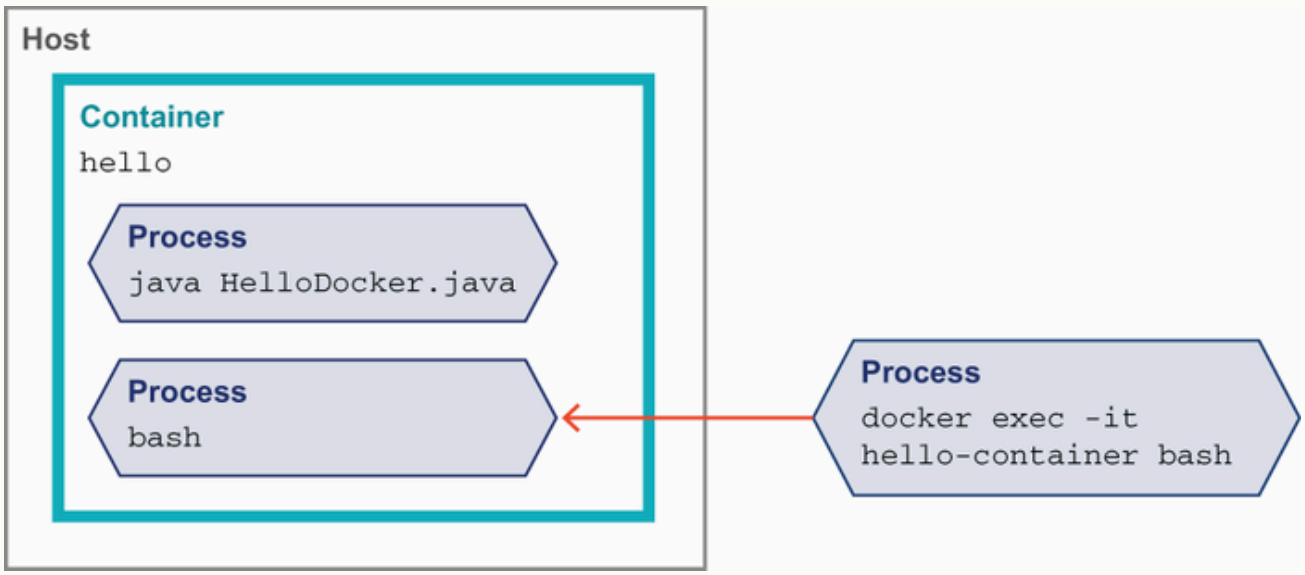


Figure 12.8 `docker exec` into a container

We aren't limited to just basic Unix commands, though. For instance, we can use `jps` and `jcmd` to inspect the running JVMs in our container as follows:

```
root@18a5f04bb4c8: jps
1 Main
```

```
148 Jps

root@18a5f04bb4c8: jcmd 1 VM.version
1:
OpenJDK 64-Bit Server VM version 17.0.1+12
JDK 17.0.1
```

In chapter 7, we explored the deep visibility we can gain with the JFR (JDK Flight Recorder) tools. With a shell into the running container, we can gather JFR data with a few simple commands. If it's not already running, we tell JFR to start recording like this:

```
root@4f146639fcfc: jcmd 1 JFR.start
1:
Started recording 1. No limit specified, using maxsize=250MB as default.
```

After we've let the application gather data for a while, we save the current recording to a file inside the container as follows:

```
root@4f146639fcfc: jcmd 1 JFR.dump name=1 filename=./capture.jfr
1:
Dumped recording "1", 293.3 kB written to:
```

To inspect the file offline, we'll need to copy it out of the container. Back on our host system, we can do this with the `docker cp` command, as shown in the next code sample. Again, our container name comes in handy to specify where to grab the file from:

```
$ docker cp hello-container:/opt/app/bin/capture.jfr .  (1)
```

(1) The first parameter to cp is the file source, and the second, its destination. It's specified with the format container-name:path. The second parameter is local, so we don't need a container name and just use the path.

`capture.jfr` is now available on your local system to open via JDK Mission Control (JMC).

Because Docker exposes an API, your `docker` commands can be pointed at a remote host instead of your local environment. See the Docker documentation at https://docs.docker.com/ for details on how to configure that.

All of these shells and command-line options are good for getting at the low level of what's happening in our containers. But what if we just want a breakpoint in our IDE for the Java application in a local container? Fortunately the JDK's remote debugging facilities have all the pieces we need to configure this, as shown next:

```
docker run --rm \
  -p 8090:8090 \
  -e JAVA_TOOL_OPTIONS=\
  '-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address=*:8090' \
  --name hello-container \
  hello
```

In addition to the normal output of our application starting up, you should see a message like this indicating the remote debugging port is available:

```
Listening for transport dt_socket at address: 8090
```

From here you can use your IDE features to debug a Remote JVM, pointed at port 8090. Everything should behave much like debugging the application on your local environment, but all from the cozy, bounded world of the container.

2.3.7 Logging with Docker

As we've seen repeatedly, the separation that containers introduce from their host environment requires a change of mind-set. One common stumbling block is logging. Whether you use one of the popular logging frameworks or simply write to `System .out`, it's common for a service to produce output when running. We don't want to lose access to this information just because we've moved into a container.

You can take a manual approach with techniques we've already seen in this chapter. Simply write your logs to disk as before. When you need to examine them, you can use `docker exec` or `docker cp` to access to the files like this:

```
// Start our container
$ docker run --rm --name hello-container hello

// In another shell, copy the file locally
// Assumes log is at /log/application.log
$ docker cp hello-container:/log/application.log .
```

```
// Or alternatively, tail the file continually
$ docker exec hello-container tail -f /log/application.log
```

However, this introduces some friction in retrieving this information—and the potential for data loss if the container is fully removed prematurely.

A common practice—with or without containers—is to forward logs from the application to a central location. The destination of this forwarding could be just centralized storage, an indexing service such as Elasticsearch, or even a fully external logging provider.

If we try to keep the simple practice of writing to a local file in our containers, though, we must answer the question of where our log-forwarding application runs. Putting it inside the containers consumes additional memory and resources we need to account for, and generally it's recommended to avoid having multiple things within a single container. Containers allow for mounting a volume so log files could be shared between the containers and host, but this requires configuration and isn't always performant.

A better alternative is to lean on the fact that Docker captures anything our containers write to the typical output streams, `STDOUT` and `STDERR`. On the host, these streams are saved to well-known file locations for every container being run. This simplifies configuration because we can install log forwarding once on the host and just tell our individual containers to write to `STDOUT` instead of files. It's also compatible with existing logging libraries such as `log4j2`, which has appenders to write to `CONSOLE` for exactly this purpose.

This type of infrastructure setup around how to run your containers and capture their logs is an example of issues that come with scaling containers beyond a single host. Providing a systematic way to address such questions is one of the key benefits of our next topic: Kubernetes.

12.4 Kubernetes

This introduction to Docker really only scratches the surface of configuring and customizing our containers. In a real production environment, you may need many instances of your containers. Managing large fleets of containers with just `docker` commands quickly gets out of control, and it's not uncommon for a production environment to have hundreds of separate containers. You need to automate these tasks. The general term for such automation is an *orchestrator*, and although there are many options in the field, Kubernetes is the dominant solution.

Kubernetes (often referred to as K8s) is an open source project originally derived out of Google's internal work on container orchestration. At its heart, it provides standard, API-driven tools to describe the desired state for a system and then ensure that state is maintained over time.

Kubernetes models your system as a set of *objects* of different types. A set of *controllers* run continually, watching the actual state of the system and applying changes (such as creating a new container if an old one dies) so the desired and actual state of the system match.

A full treatment of Kubernetes is far beyond the scope of this book, but to get a taste of how it works, let's look at the most basic object types and how we'd use them with the container skills we've gotten so far.

- *Cluster*—A single installation of Kubernetes on anything from a single machine to many hundreds of nodes
- *Node*—A single machine (virtual or physical) in the cluster
- *Pod*—A deployable unit of one (or more) containers
- *Deployment*—A declarative way to deploy a pod
- *Service*—An object exposing containers in the cluster to callers

To step through these ideas and demonstrate we'll use `minikube`, a local development environment from the Kubernetes project itself. See the linked instructions for current installation instructions on your OS (**https://minikube.sigs.k8s.io/docs/start/**).

Once installed we can start a local cluster with the command `minikube start`, as shown next. Note this may take several minutes the first time to download all the required images:

```
$ minikube start

  minikube v1.25.2 on Darwin 11.6.2
  Using the docker driver based on existing profile
  Starting control plane node minikube in cluster minikube
  Pulling base image ...
  Downloading Kubernetes v1.23.1 preload ...
  > preloaded-images-k8s-v17-v1...: 504.44 MiB / 504.44 MiB  100.00%
  Restarting existing docker container for "minikube" ...
  Preparing Kubernetes v1.23.1 on Docker 20.10.12 ...
  * kubelet.housekeeping-interval=5m
  Verifying Kubernetes components...
  * Using image kubernetesui/dashboard:v2.3.1
```

```
* Using image kubernetesui/metrics-scraper:v1.0.7
* Using image gcr.io/k8s-minikube/storage-provisioner:v5
Enabled addons: storage-provisioner, default-storageclass, dashboard

Done! kubectl is now configured to use "minikube" cluster and "default"
namespace by default
```

Our Kubernetes cluster is now running locally. We can stop the cluster with the `minikube stop` command or, if we're entirely finished experimenting, remove it with `minikube delete`.

Although Kubernetes provides a REST API for systems to interact with, there's a convenient wrapper more suitable for human consumption via the `kubectl` command. We can use this to view, create, and edit the objects in our cluster. For instance, `minikube` takes care of creating our node objects by default, but we can `kubectl describe node` to see what it set up on our behalf. This listing highlights only a few portions of the output as it provides a lot of detail:

```
$ kubectl describe node

Name:               minikube
Roles:              control-plane,master
Labels:             kubernetes.io/arch=amd64
                    kubernetes.io/hostname=minikube
                    kubernetes.io/os=linux
Addresses:
  InternalIP:  192.168.49.2
  Hostname:    minikube

Non-terminated Pods:          (12 in total)                    (1)
  Namespace                   Name
  ---------                   ----
  kube-system                 coredns-64897985d-n8fzv
  kube-system                 etcd-minikube
  kube-system                 kube-apiserver-minikube
  kube-system                 kube-controller-manager-minikube
  kube-system                 kube-proxy-4zvll
  kube-system                 kube-scheduler-minikube
  kube-system                 storage-provisioner
  kubernetes-dashboard        dashboard-metrics-scraper-58549894f-bcjh4
  kubernetes-dashboard        kubernetes-dashboard-ccd587f44-mq8zv

Events:                                                      (2)
```

```
Type   Reason                   Message
----   ------                   -------
Normal Starting                 Starting kubelet.
Normal NodeHasSufficientMemory Node status is: NodeHasSufficientMemory
Normal NodeHasNoDiskPressure   Node status is: NodeHasNoDiskPressure
Normal NodeHasSufficientPID    Node status is: NodeHasSufficientPID
```

❶ Kubernetes itself runs in pods on the node, listed here.

❷ Events can be helpful when debugging if unexpected problems occur in a node.

With `minikube` providing our cluster and node, we're ready to run some software. To keep things simple we'll use the `k8s.gcr.io/echoserver:1.4` image, which as the name suggests just echoes back information about HTTP requests sent to it.

NOTE `minikube` supports working with local images, but it runs a separate Docker daemon, so the image management gets a little more complex. Consult the README at https://github.com/kubernetes/minikube if you are looking to do more local development using `minikube`. We'll stick to published images in our examples to keep it simple.

Our first goal is to get a pod on the cluster running the `echoserver` container. We do that by asking `kubectl` to create a deployment, as shown in the next code snippet. The deployment object tells the Kubernetes cluster the desired state of having our pod running. Kubernetes's control loop notices the desired state doesn't match reality and start our pods for us to address that:

```
$ kubectl create deployment echoes --image=k8s.gcr.io/echoserver:1.4
deployment.apps/echoes created
```

We can inspect the cluster to see that our deployment exists using the standard `kubectl get` command, as follows. This command works across any type of object in the system:

```
$ kubectl get deployments
NAME    READY   UP-TO-DATE   AVAILABLE   AGE
echoes  1/1     1            1           55s
```

If we look for pods, we'll also see shortly that the cluster has aligned its actual state with what our deployment requested, as shown here. Figure 12.9 gives a visual of the

single pod state we've reached.

```
$ kubectl get pods
NAME                 READY   STATUS    RESTARTS   AGE
echoes-7989cff4bc-7m4df   1/1     Running   0          78s
```

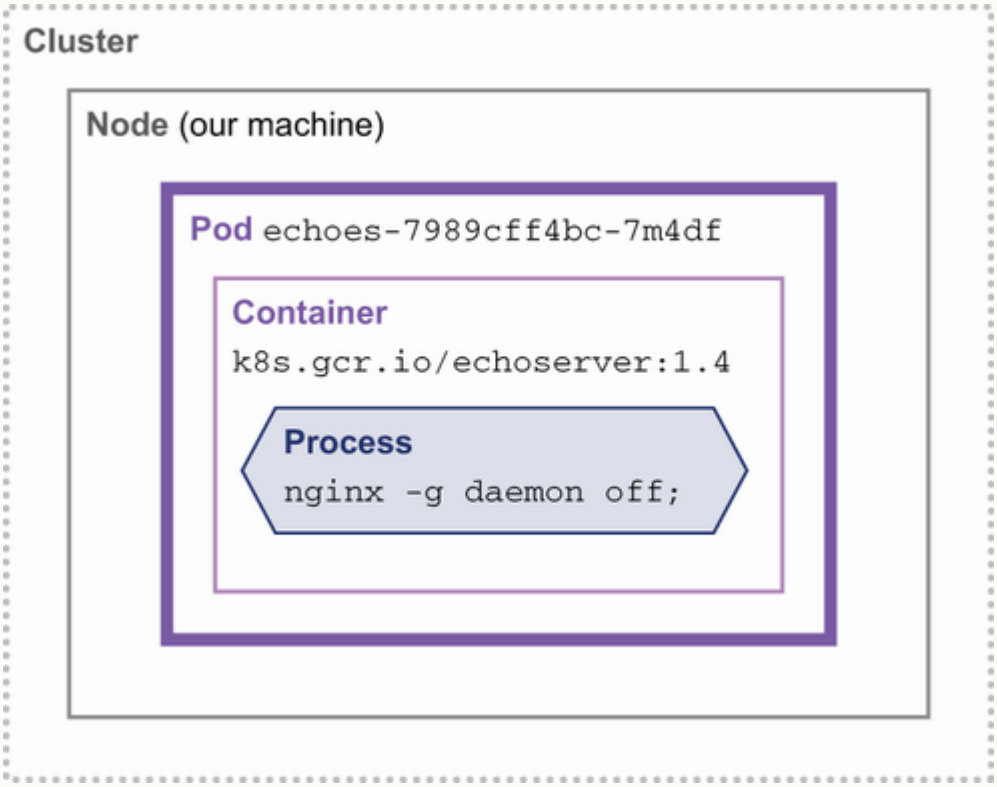


Figure 12.9 Kubernetes cluster with one pod running

The `kubectl create deployment` command is an easy way to get started, but its arguments only scratch the surface of what Kubernetes can configure. The natural representation of a Kubernetes object is given in YAML, and we can access the full picture via `kubectl edit deployment echoes`, as shown in the next code sample. This will open your default editor with the object's current YAML. If you make changes to the file, they are applied when the editor exits. We won't discuss all of these options, so refer to the documentation at **http://mng.bz/M5m2** for further information:

```
# Please edit the object below. Lines beginning with a '#' will be ignored,
# and an empty file will abort the edit. If an error occurs while saving
# this file will be reopened with the relevant failures.
#
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations:
    deployment.kubernetes.io/revision: "1"
```

```
  creationTimestamp: "2022-02-01T08:26:32Z"
  generation: 1
  labels:
    app: echoes
  name: echoes                                    (1)
  namespace: default
  resourceVersion: "1310"
  uid: e8b775f6-243e-46c1-9275-dadaecf2db3b
spec:                                             (2)
  progressDeadlineSeconds: 600
  replicas: 1                                     (3)
  revisionHistoryLimit: 10
  selector:
    matchLabels:
      app: echoes
  strategy:
    rollingUpdate:
      maxSurge: 25%
      maxUnavailable: 25%
    type: RollingUpdate
  template:
    metadata:
      creationTimestamp: null
      labels:
        app: echoes
    spec:
      containers:
      - image: k8s.gcr.io/echoserver:1.4          (4)
        imagePullPolicy: IfNotPresent
        name: echoserver
        resources: {}
        terminationMessagePath: /dev/termination-log
        terminationMessagePolicy: File
      dnsPolicy: ClusterFirst
      restartPolicy: Always
      schedulerName: default-scheduler
      securityContext: {}
      terminationGracePeriodSeconds: 30
status:                                           (5)
  availableReplicas: 1
  conditions:
  - lastTransitionTime: "2022-02-01T08:26:33Z"
    lastUpdateTime: "2022-02-01T08:26:33Z"
    message: Deployment has minimum availability.
    reason: MinimumReplicasAvailable
    status: "True"
```

```
    type: Available
  - lastTransitionTime: "2022-02-01T08:26:32Z"
    lastUpdateTime: "2022-02-01T08:26:33Z"
    message: ReplicaSet "echoes-7989cff4bc" has successfully progressed.
    reason: NewReplicaSetAvailable
    status: "True"
    type: Progressing
  observedGeneration: 1
  readyReplicas: 1
  replicas: 1
  updatedReplicas: 1
```

❶ The echoes name we gave our deployment

❷ spec describes the desired state of our deployment.

❸ An important value we'll discuss in a moment that determines the number of pods we want running

❹ The image we requested for our pods to run

❺ status tells us what's currently observed about the state of the deployment. Note that it also has replicas, telling us how many are seen running now.

What happens if we change the `spec` value of `replicas: 1` to `3`? Kubernetes will see the mismatch between the deployment state and what's actually on the cluster and start new containers on our behalf, as shown next. Figure 12.10 shows the result after the containers have had a chance to start up.

```
$ kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
echoes-7989cff4bc-7m4df   1/1     Running   0          7m38s
echoes-7989cff4bc-7qn47   1/1     Running   0          8s
echoes-7989cff4bc-cmngm   1/1     Running   0          8s
```

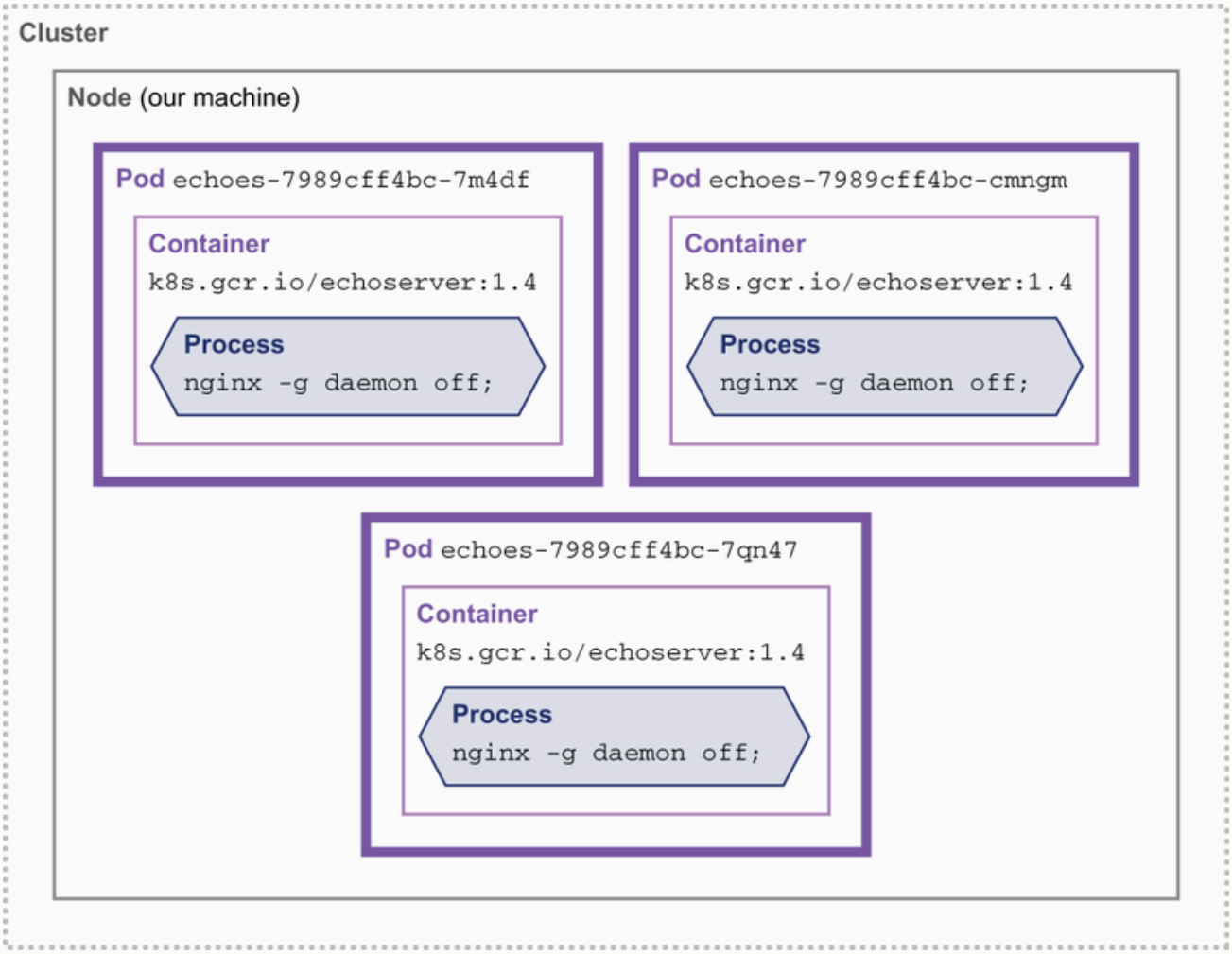


Figure 12.10 Kubernetes cluster with multiple pods running

In practice, you likely won't be hand-editing YAML files on a production Kubernetes cluster, but all the tooling builds on this—CI/CD systems, generated or source-controlled Kubernetes manifests—are just helpers creating the right YAML and API calls for us.

On our local system, the same tricks we played before with `docker ps` and `docker exec` will work to let us take a closer look at the running containers. Once we know the container name, `kubectl` has a slightly cleaner command for letting us start a shell inside the pod, as shown here:

```
$ kubectl exec echoes-7989cff4bc-7m4df -- bash
root@echoes-7989cff4bc-7m4df: uname -a
Linux echoes-7989cff4bc-7m4df 5.10.76-linuxkit #1 SMP \
  Mon Nov 8 10:21:19 2021 x86_64 x86_64 x86_64 GNU/Linux
```

One last step is needed to make this deployment more useful. By default we can't talk to the pods in the cluster at all. If we look at the `docker ps` for the containers, you'll see that no ports are exposed.

Kubernetes addresses this via its *service* abstraction, which is a general interface for working with various load balancing and traffic routing into the cluster. The details here quickly get beyond the scope of this introduction, but we'll set up the simplest of them, called a `NodePort` with `kubectl expose`, as follows:

```
$ kubectl expose deployment echoes --type=NodePort --port=8080
service/echoes exposed
```

As seen in figure 12.11, this creates a new object in the cluster representing our `NodePort`.

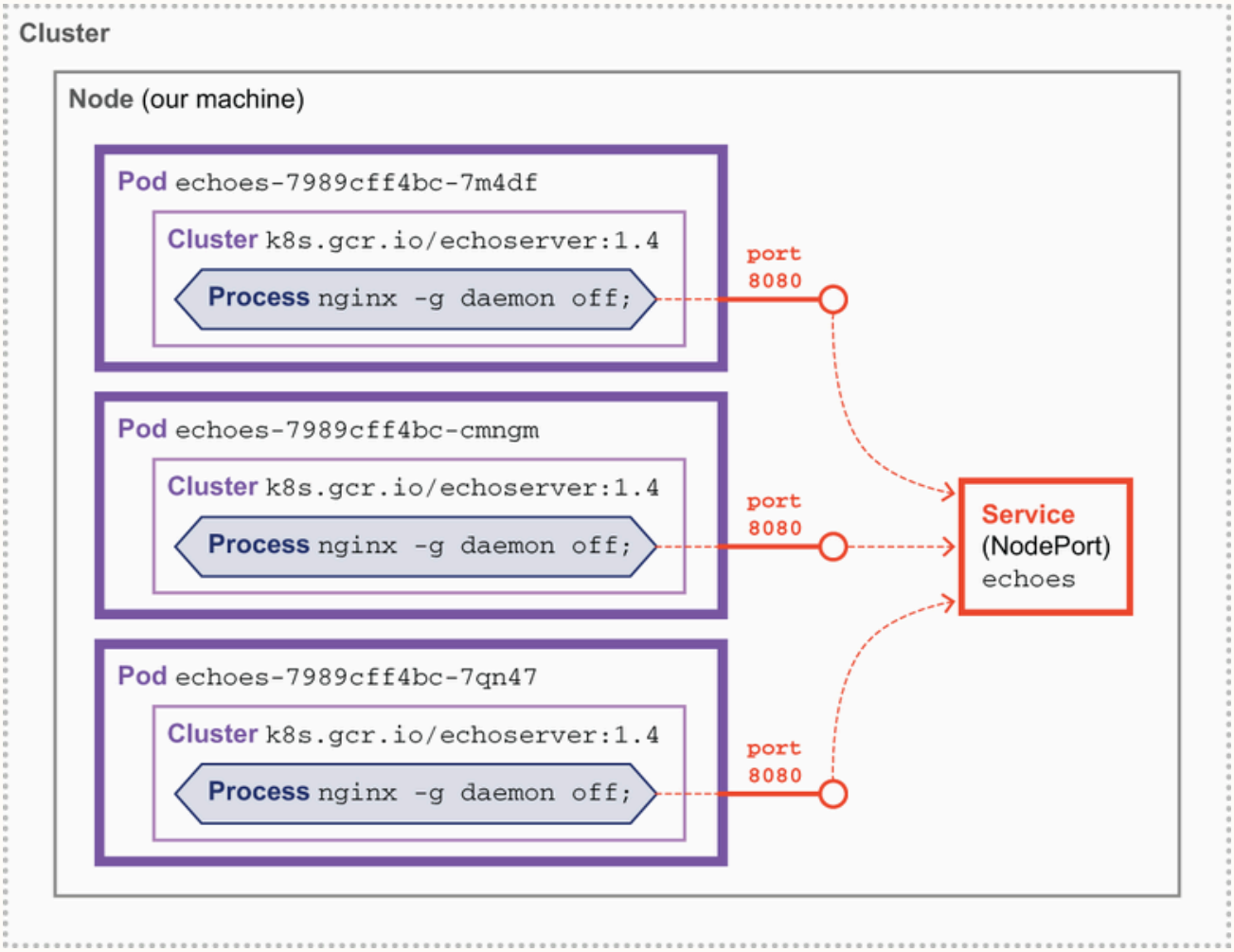


Figure 12.11 `NodePort` and services in the Kubernetes cluster

Viewing the services, we can see a `NodePort` configured in the cluster, as shown next:

```
$ kubectl get services
NAME       TYPE       CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
echoes     NodePort   10.108.182.100  <none>        8080:31980/TCP 12s
kubernetes ClusterIP  10.96.0.1       <none>        443/TCP        35m
```

Internally, this means that port 8080 is available on every node in the cluster and will forward traffic to our pods. Now that we have a way for traffic to reach our pods, we still need to reveal this outside the cluster to be able to call it. `kubectl` supports this with its port-forwarding feature like this:

```
$ kubectl port-forward service/echoes 7080:8080
Forwarding from 127.0.0.1:7080 -> 8080
Forwarding from [::1]:7080 -> 8080
Handling connection for 7080
```

With this forwarding running in a terminal, we can visit 127.0.0.1:7080 in the browser, and we'll see our request echoed back. Figure 12.12 shows the flow of traffic through the various components to the pods.

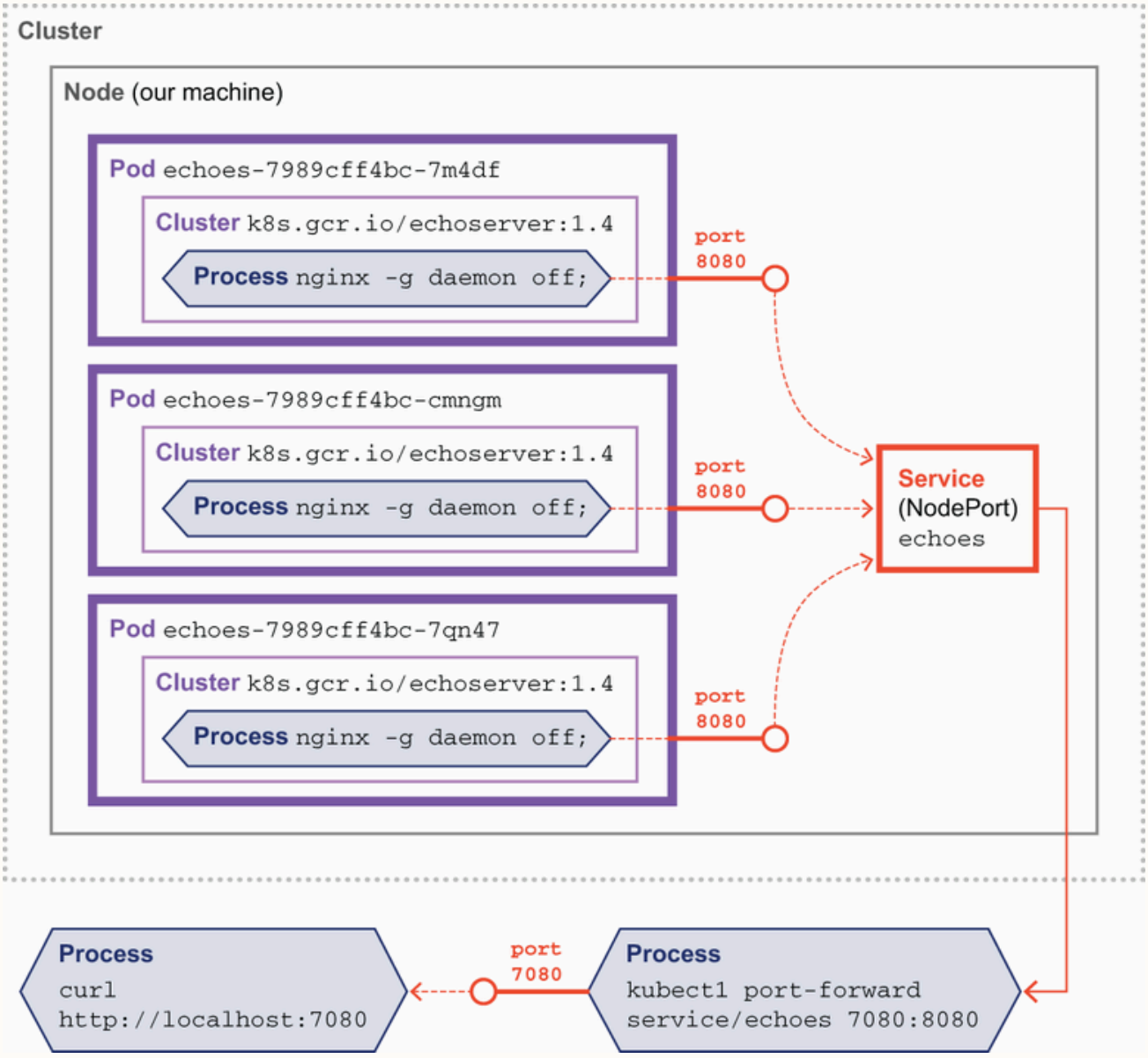


Figure 12.12 Port-forwarding into our Kubernetes cluster

Kubernetes is a large step up in complexity from just running our Docker containers locally, but it also provides a solution to the difficulties of running containers at scale. Whether or not we're running on Kubernetes, though, we almost always care about the performance of our services. Let's examine how to manage how well our containers are running in production.

12.5 Observability and performance

Java technology was originally designed for a world where JVMs ran on bare metal in data centers, and where the developer could remain relatively isolated from (or even ignorant of) the details of the deployment environment. The world is changing at a fundamental level, however. Cloud native deployments—especially containers—are here and are being adopted quickly (but at varying rates across different parts of the industry).

Containers present some particular challenges for understanding the details of what is happening in a modern application. For example, it is not common practice for containers to run services such as an ssh daemon, making it impossible to log in to a container to observe what is happening. Instead, any data about the health of applications must be exported out of the container.

The DevOps practice known as *observability* grew out of several separate strands of modern development practice, including both the application performance monitoring (APM) space and the need for visibility into orchestrated systems, such as Kubernetes. It aims to provide highly granular insights into the behavior of systems along with rich context. The techniques it provides are very useful, if not essential, for understanding and tuning the performance of Java applications in containers.

2.5.1 Observability

Overall, observability is fairly simple set of concepts:

1. Instrument systems and applications to collect relevant data
2. Send this data to a system that can store and analyze it (including query capability)
3. Provide visualizations and insights into systems as a whole

The query and visualization capability is the key to the power of observability. It has been described as being the ability to "Get answers to questions that you didn't know you'd need to ask"—and this is possible only via the collection of sufficient data to accurately model the system's internal state.

NOTE The theory behind Observability comes from system control theory—essentially the question: "How well can the internal state of a system be inferred from outside?"

Ultimately, the goal is to be able to obtain *actionable* insights from, and about, the entire system. This should replace fragmentary views that are based on just one or two pieces of the overall system.

So, while incident resolution is an obvious use case that is a good fit for observability—it is where the practice originated after all—it is also true that the potential domain of applicability is much larger. If the right data is being collected, the stakeholders for observability are much broader than just software reliability engineers (SREs), production support, and DevOps folks.

Observability is particularly relevant to containerized applications, because these deployments tend to be more complex than traditional on-premises applications. There are typically more services and components in cloud-deployed application, with more complex topology as well as a much faster pace of change (driven by practices such as continuous deployment).

This is also combined with the increasing popularity of new Cloud native technologies that have new operational behaviors. This includes Kubernetes, as well as Function as a Service deployments, such as AWS Lambda. This new world makes root cause analysis and incident resolution potentially a lot harder.

Observability data is often conceptualized in terms of "three pillars." This is a simple mental model (some would argue *too* simple) but is useful for developers who are new to observability. The pillars follow:

- *Distributed traces*—Records of a single service invocation, corresponding to a single request from a user
- *Metrics*—Values measuring specific activity over a time interval
- *Logs*—Immutable records of discrete events that happen over time (can be plain text, structured, or binary)

The core libraries and instrumentation components are all open source, and most of them are managed through industry bodies such as the Cloud Native Compute Foundation (CNCF).

OpenTelemetry

The OpenTelemetry project (**https://opentelemetry.io/**), a major project within CNCF, is a set of standards, formats, client libraries, and associated software components for providing observability. The standards are explicitly cross-platform and not tied to any particular technology stack.

This provides a framework that integrates with OSs and commercial products and can collect observability data from apps written in many languages. Because the implementations are open source, they are of varying levels of technical maturity, depending on the interest that OpenTelemetry has attracted in the particular language community.

OpenTelemetry came from the merger of two prior open source projects, OpenTracing and OpenCensus projects. Although OpenTelemetry is still maturing, it is gaining momentum and an increasing number of applications and teams are investigating and implementing it. This number seems set to grow significantly during 2022 and 2023.

From our perspective, the Java/JVM implementation is one of the most mature available and has a number of advantages over traditional APM/monitoring. In particular, the use of an open standard provides the following:

- Vastly reduced vendor lock-in
- Open specification wire protocols
- Open source client components
- Standardized architecture patterns
- Increasing quantity and quality of open source backend components

OpenTelemetry has several subprojects that make up the standard as a whole, and they are not all at the same level of maturity in terms of their overall lifecycle.

The Distributed Tracing specification is at v1.0 and is being actively deployed into production systems. It replaces OpenTracing completely, and the OpenTracing project has been officially archived. The Jaeger project, one of the most popular distributing tracing backends, has also discontinued its client libraries and will default to OpenTelemetry protocols going forward.

The OpenTelemetry Metrics project is not quite as advanced but has reached v1.0 and general availability (GA). At time of writing, the protocol is stable, and the API is at feature-freeze.

Finally, the Logging specification is still in draft and is not expected to reach v1.0 until late 2022. There is still acknowledged to be a certain amount of work to do on the spec.

Overall, OpenTelemetry as a whole will be considered to be v1.0/GA when the Metrics standard reaches v1.0 alongside Tracing.

The Java libraries for OpenTelemetry can be deployed into your application using either manual methods (where the developer must consciously choose which parts of the application need to be instrumented) or the use of automatic instrumentation (using a Java agent). Java components for OpenTelemetry can be found on GitHub and live in several projects, including **http://mng.bz/aJyJ**.

A full discussion of how to implement a full observability solution (whether based on OpenTelemetry or another stack) is outside the scope of this book, but the well-grounded Java developer would be well advised to explore this area thoroughly.

Related to observability are some performance subtleties that engineers who are not Java/VM specialists may not be aware of. Let's take a closer look.

2.5.2 Performance in containers

Many developers, when migrating their Java applications into containers, will try to use the smallest possible containers. This seems to make sense, because cloud-based applications are typically charged by the amount of RAM and CPU that they use.

However, the JVM is a very dynamic platform, and certain important parameters are automatically determined by the JVM at startup time, based on the observed properties of the machine the JVM is running on.

These properties include the type and count of the CPUs and the physical memory. The behavior of the running application can and will be different when running on differently sized machines—and this includes containers. Some of these dynamic properties follow:

- JVM Intrinsics, a JIT technique that can make use of very specific CPU features (e.g. vector support)
- Sizing of internal threadpools (such as the "common pool")
- Number of threads used for GC

Just from this list, we can see that incorrectly choosing the size of the container image can cause problems related to GC or common thread operations. However, the problem is fundamentally deeper than this.

Current versions of Java, including Java 17, perform some dynamic checks and decide the GC to use *ergonomically* (automatically), if a GC is not explicitly specified on the command line. If you didn't specify a collector, then the logic is as follows:

- If the machine is "server class," choose G1 (Parallel for Java 8).
- If the machine is not "server class," then choose Serial.

The working definition of a server class machine is: >= two physical CPUs and >=2 GB of memory

This means that if a Java application is run on a machine that appears to have less than two CPUs and 2 GB of memory, then unless a specific collector algorithm is explicitly chosen, the Serial algorithm will be used. This is usually not what teams want—and leads to the following best practice:

TIP Always run Java applications in containers with at least two CPUs and 2 GB of memory.

It is also important to recognize that the traditional Java application lifecycle consists of a number of phases: bootstrap, intense class loading, warmup (with JIT compilation), and then a long-lived steady state (lasting for days or weeks) with relatively little class loading and JIT. This model is challenged by cloud deployments where containers may live for much shorter time periods and cluster sizes may be dynamically readjusted.

In this new world, Java has to ensure that it remains competitive along several key axes, including the following:

- Footprint
- Density
- Startup time

Fortunately, there is ongoing work and research into making sure that the platform continues to optimize for these characteristics—we'll hear more about it in chapter 18.

Summary

- Containers have radically changed how we package and deploy applications and require some new techniques and ideas.
- Containers represent another layer of abstraction on top of the classic operating systems, hypervisors, and VMs that we've seen in the past.

- Docker is the most common tool for building, publishing, and running container images.
- We specify a container image via a `Dockerfile`. The resulting image contains our application and a complete environment in which it can run, including the JVM, native dependencies, and additional tooling.
- Containers introduce an additional layer to networking in particular. In its most basic form, we have to manage the ports that our containers expose for them to be accessible to the outside world.
- Running fleets of containers at scale is a lot to keep track of, so orchestrators are used to do that systematically. The most popular choice for this is Kubernetes.
- Kubernetes provides a rich, extensible API for declaring the desired state of your system and bringing that to life at runtime. It is accessible via the command line and a REST API with an enormous ecosystem of supporting tools around it.
- A key feature of containers is enforcing constraints around resources. These limits on memory and CPU can have performance implications for your application running in a container.

Previous chapter
< 11 Building with Gradle and Maven

Next chapter
13 Testing fundamentals >